16/02/2022

FUNDAMENTALS OF OOPS

ENCAPSULATION:

The process of Binding both states and Behaviour of an object together is known as Encapsulation.

We can achieve encapsulation with the help of class.

Eg: Let us consider an Object Marker which has following states:

~~class Marker~~ static String Comp-name;

String colour;

double price;

It has behaviours:

```
displayDetails() {
    ========
    ========
}

write()
{
    ========
    ========
}

etc..
```

Combining above states as well as Behaviour is possible with the help of class as shown below:

```
class Marker
{
    static String comp-name = "CELLO";
    String colour;
    double price;
    void displayDetails()
    {
    System.out.println("Company name :" + comp-name);
    System.out.println("Colour of Marker :" + colour);
    System.out.println("price of Marker :" + price);
    }
    void write()
    {
    System.out.println("I am used to write");
    }
}
```

▶ DATA HIDING:

The process of restricting the direct access to the states of the class but providing controlled access with the help of behaviours of the same class is known as Data hiding

Data hiding is possible only inside encapsu-ted class.

▶ STEPS TO ACHIEVE DATA HIDING:

1) Prefix the data Member with the help of private access modifier.

2) Define the getter & setter Method based on Requirement

▶ PRIVATE:

1) It is a keyword

2) It is a access modifier

3) Any member which is prefixed with private access modifier can be accessed only within the

same class.

GETTER METHOD:

Getter Method is a method which is used to print the private member's data from different class

⇒ It returns the data of members which has to be read

SYNTAX:

```
Datatype of private Member   getPrivateMemName ( )
{
return Private_member_name;
}
```

Ex: double getBal ( )
{
return bal;
}

▶ SETTER METHOD :

Setter method is used to modify the data of private member of different class.

Setter method must always accept an input, must have formal arguements.

SYNTAX :

```
void    setPrivateMemberName (Datatype of Pr.Mem id
{
this.PrivateMemberName = identifier;
}
```

Eg:
```
void setBal (double bal)
{
this.bal = bal;
}
```

NOTE : We can achieve the flexibility of accessing the private member's data in following ways:

1) Readable

2) Readable & Modify.

3) Only Modify

4) Not Readable & Not Modifiable

Eg:

```
class SavingsAccocent
{
int acno;
String name;
privatc double bal;
String ifsc;
SavingsAccount()
{
}.
Savings Accocent(int acno, String name, double bal, String ifsc)
{
this.acno= acno;
this.name = name;
this. bal= bal;
this. ifsc= ifsc;
}
void displayDetails()
{
System.out.println("Accocent ncember :" + acno);
System. out. println ("Accocent holders name :" + name);
System. out. println ("Accocent balance :" + bal);
System.out.println ("Branch ifsc code : " + ifsc);
System. out. println ("==========");
}.
//to deposit
double deposit (double amt)
{
.bal + = amt;
System.out.println("amt+" is successfully deposited to
your accocent");
```

```
return bal;
}

// to withdraw

double withdraw (double amt)
{ if(amt < bal)
  {
bal- = amt;
System.out.println("amt+" is debited from your acco
return bal;
}
else
{
System.out.println("Insufficient balance");
return bal;
}
}
}

class SADriver
{ public static void main(String[] args) {
SavingsAccount a1 = new SavingsAccount(101, "Sheela", 5c
                                        "IIIC1001")
a1.displayDetails();
System.out.println(a1.bal); //CTE → private member.
}
}
```

1) To make 'bal' private member Readable add getter method.

```
double getBal()
{
return bal;
}
```

2) To make 'bal' private member both Readable & able to Modify:

```
double getBal()
{
return bal;
}
~~double~~ void setBal(double bal)
{
this.bal = bal;
}
```

3) To make 'bal' private member only ~~it~~ to modify

```
void setBal(double bal)
{
this.bal = bal;
}
```

4) Not Readable & not able to modify:

NOTE: No need of writing getter & setter methods.